

Case Study: Scientist-Facing Exploration of Differential Gene Expression

A Forward Deployed Engineering exercise — from a raw bioinformatics output to a scientist-facing product concept, without touching a line of application code first.

Target roles: Forward Deployed Engineer, Solutions Architect (Scientific/R&D), Research Systems Engineer, Scientific Platform Engineer, Translational Bioinformatics Engineer, Applied AI/ML Engineer.

The one-line pitch

Given nothing but an existing DGE (differential gene expression) results table — the kind of file that lands in a scientist’s inbox after every RNA-seq experiment — I ran a full FDE discovery process: understand the data, understand the scientist, define the product, check the data against the product, design the flow, and spec the interface. No feature was designed before the data or the user could justify it.

Why this project, and why it’s shaped like this

Most portfolio projects in this space are “I built a Streamlit app that plots a volcano plot of my DGE data.” That demonstrates you can use pandas and plotly. It does not demonstrate the skill these roles actually hire for: **sitting between a scientific workflow and an engineering org, and building the translation layer between them.**

It’s worth being direct about something most of these portfolio projects get backwards, including earlier drafts of this one: **the valuable skill here isn’t extracting insight from a dataset — it’s knowing which questions to ask a domain expert so that extracting insight from the dataset becomes mostly unnecessary.** None of the roles above are data scientist roles. A client conversation that surfaces “this is one comparison among five we’re planning” or “our scientists don’t trust padj under 3891 missing values without an explanation” does in five minutes what several passes of pandas analysis can only approximate. The data analysis in 03 is real and it’s useful, but in this project it’s standing in for a conversation with a research team that doesn’t exist. **02_discovery_notes.md is the document that makes that substitution explicit** — it’s the set of questions that, asked early, would have shortcut most of the data-analysis work, and it’s arguably the single most representative artifact in this repo for what these roles actually screen for.

So instead of starting with a UI, I started with the mental model an FDE actually uses:

```
CLIENT → existing scientific workflow → existing data output →  
understand data and its context → ask what scientists need to do →  
identify friction → translate friction into product needs →  
build prototype → test with users → iterate
```

Then I worked that loop end-to-end against a real dataset (the classic `airway` DESeq2 result — human airway smooth muscle cells, dexamethasone vs. untreated, ~22,000 genes) as if a pharmaceutical respiratory research team had just handed it to me.

What’s actually in this project

#	Document	What it answers
00	FDE Mental Model	What does an FDE actually <i>do</i> ?
01	Project Brief	What is the client asking for, scoped like a real internal brief?
02	Discovery Notes	What would I ask the client directly, before touching the data?
03	Data Exploration Findings	What does this dataset actually contain?
04	FDE Data Reasoning	What does each measurement <i>mean</i> , and what does that imply for the product?
05	Scientist Workflow	What does a researcher actually do with a DGE table, stage by stage?
06	Product Requirements	What should the application do — and explicitly not do?
07	Data-to-Feature Mapping	What can this data support today vs. what needs more data from the client?
08	Application User Flow	How does the scientist move through the product, screen to screen?
09	UI Prototype Specification	What does each screen actually look like, and why?
10	Prototype Plan	What’s the concrete build plan — data model, screens, build order?
11	Data Specification	What’s the exact data contract the application is built on?
12	Application Architecture	What’s the technical architecture — stack, layers, state, components?
13	Component Specification	What are the actual React components, and what does each one do?
14	Implementation Plan	What’s the exact build sequence — milestones, order, definition of done?

Doc 02 sits deliberately right after the brief, ahead of any data work — see below for why.

Full documents are in [docs/](#). This file is the narrative wrapper for anyone reading the project as a portfolio piece rather than working through it artifact-by-artifact.

The core insight the whole project is built on

A DGE result has five columns — `baseMean`, `log2FoldChange`, `lfcSE`, `pvalue`, `padj` — and it’s tempting to treat that as “just a table to search and filter.” The data itself argues against that:

- **91.5%** of genes sit within ± 0.5 log2FC — the “interesting” genes are a small minority inside a large, mostly-uninteresting background.
- Effect size and statistical evidence are genuinely different dimensions: 13,535 genes have small effects and weak evidence, but 503 genes have moderate effects (0.5–1 log2FC) with extremely strong evidence ($\text{padj} < 0.001$) — a naive “sort by fold change” view would bury them.
- ~18% of `padj` values are missing, and treating “missing” as “not significant” would be a quiet but serious scientific error.

None of that is visible from a spec meeting. It only shows up once you actually open the data. That’s the FDE muscle this project is meant to demonstrate: **the product decisions came from inspecting the data and modeling the scientist’s workflow, not from assuming what a bioinformatics dashboard “should” look like.**

Process, condensed

1. **Ask before analyzing** (02) — wrote out the discovery questions a real engagement would open with: scope and definition of done, experimental context, which annotation sources the team already trusts, what happens downstream of a flagged gene, and what level of trust the tool needs to earn. This is the step that’s easy to skip when there’s no client in the room, and

skipping it is exactly what makes the next step necessary.

- 2. **Understand the data** (03 , 04) — with no client to ask, ran the dataset through pandas instead, characterized response magnitude, statistical evidence, expression, and uncertainty, and used those findings to reframe the problem from “*scientists need to search 22,000 genes*” to “*scientists need to orient themselves in a multidimensional result before they know what to search for.*”
- 3. **Understand the scientist** (05) — modeled the workflow a researcher actually follows after receiving a DGE table: orient → understand the response → notice patterns → explore → narrow → investigate a gene → add biological context → decide what deserves follow-up. Each stage names the friction that exists today (spreadsheets, static plots, switching between tools) and the opportunity for a product to remove it.
- 4. **Define the product** (06) — turned that workflow into explicit requirements, with explicit product boundaries: the system surfaces evidence, it does not declare a gene “important,” and it never silently converts a missing value into a negative result.
- 5. **Check requirements against reality** (07) — the step most portfolio projects skip. For every requirement: can this be built from the five columns we have (Type A), does it need to be derived (Type B), or does it need data the client hasn’t given us yet (Type C — e.g. pathways, gene annotations, literature)?
- 6. **Design the flow, then the screens, then the full build spec** (08 - 14) — screens were designed only after the flow, organized around the scientist’s next question rather than “what can we chart from this column.” The homepage is deliberately *not* a volcano plot with a search bar — it’s an experiment-orientation screen, because that’s the question a scientist actually asks first. From there, 10 - 14 carry the spec all the way to buildable: prototype plan, data contract, technical architecture, component breakdown, and a milestone-by-milestone implementation plan with a definition of done.

What’s built vs. what’s roadmap

The brief scopes this honestly as a V1-of-9 iteration plan, which is itself part of the FDE story — knowing what to ship first and what to defer:

Version	Capability	Status
V1	Interactive exploration (Python, pandas, visualization)	This case study — spec complete
V2	Pattern discovery (scikit-learn)	Roadmap
V3	Learned representations (PyTorch)	Roadmap
V4	Similarity search (embeddings + vector search)	Roadmap
V5	Scientific evidence retrieval (RAG)	Roadmap
V6	Multi-step investigation (tool calling / agents)	Roadmap
V7	Biological relationships (knowledge graphs)	Roadmap
V8	Production deployment (APIs, Docker, AWS)	Roadmap
V9	Reliability (evaluation + observability)	Roadmap

(Update this table once the V1 prototype is actually built and deployed — see “Next steps” below.)

What this demonstrates, mapped to what these roles actually screen for

What the role needs	Where it shows up here
Ask the right discovery questions instead of brute-forcing an answer out of data	02 — explicit, and arguably the closest thing in this repo to what the job actually is day to day
Translate a fuzzy scientific ask into a scoped engagement	01 — brief written like an actual internal FDE brief, not a proposal
Get hands into the client’s actual data before designing anything	03 , 04 — pandas-based data characterization drives every downstream decision
Model the domain expert’s workflow, not just the data schema	05 — full scientist workflow with friction points named explicitly
Say no to things the data can’t responsibly support	06 , 07 — explicit product boundaries, explicit “requires additional data” list
Turn ambiguity into concrete client discovery questions	07 , section 16 — data gaps reframed as questions to ask the client
Design software around a domain workflow, not a database table	08 , 09 — “CSV structure ≠ scientist workflow” is stated as a design principle
Think in scientific-trust terms (traceability, missing data, evidence vs. interpretation)	Runs through every document — probably the single most consistent thread

Honest limitations (worth stating in an interview)

- This is a spec-and-reasoning exercise, not (yet) a deployed application — see “Next steps.”
- The dataset is a well-known public example (`airway`), not a live client engagement, so the discovery questions in 02 are necessarily hypothetical — written as what *should* have been asked, not questions that were actually answered by a real research team. In a real engagement, some of those questions would surface answers that reshape earlier documents in this repo; here, the data analysis had to serve as the best available substitute for that conversation.
- Biological context (pathways, annotations, literature) is explicitly scoped out of V1 — that’s a deliberate boundary, not an oversight, but it means the “so what does this gene *mean*” question isn’t answered yet.

Next steps (for you, not for the reader)

1. Build the V1 prototype following the milestone-by-milestone plan in 14 (React + Vite, no backend required — see 14’s Definition of Done for exactly what “finished” means).
2. Deploy it somewhere public and link it from this document.
3. Swap this “roadmap” table for a real “built / not built” table once V1 exists.
4. Add 2–3 screenshots or a short screen recording of the actual interface into this file — recruiters skim, and a picture of the Response Landscape screen will do more work than another paragraph.
5. If you want to push the FDE narrative further, pick *one* of V2–V9 and actually build a thin slice of it (pattern discovery with scikit-learn is the cheapest next step) — that turns “I planned a 9-version roadmap” into “I shipped the first extension of it.”

Full working documents: [docs/00_fde_mental_model.md](#) through [docs/14_implementation_plan.md](#).